Quantum Algorithmic Foundations

Learn • Build • Measure • Practice | Qiskit-ready one-page reference

CORE MODEL

1. Input & encoding

- Computation is modeled as binary input → computation → binary output.
- Binary strings can encode numbers, vectors, matrices, graphs, molecules, and more.
- Integer length: lg(0)=1; lg(N)=1+floor(log2 N) for N>=1.
- Complexity scales with input length n, not directly with the magnitude N.

2. Circuit cost

- Size(C) = total number of gates → sequential cost proxy.
- Depth(C) = longest dependency path → parallel cost proxy.
- Algorithms use circuit families {C_n}; t(n)=size(C_n).
- Big-O ignores lower-order details and constant factors in asymptotic scaling.

3. Polynomial vs exponential

- Polynomial: O(n^b) for fixed b>0; a first-order efficiency abstraction.
- The lesson contrasts polynomial arithmetic with much harder known classical factoring.

Uniformity note: the circuit descriptions themselves should also be efficiently generable.

REVERSIBLE CLASSICAL COMPUTATION

4. Gate dictionary

- Boolean set: AND, OR, NOT, FANOUT.
- Quantum set in slides: X,Y,Z,H,S,S†,T,T†, CNOT, measurement.
- Toffoli / CCX: |a,b,c> → |a,b,c XOR ab>; c=0 writes a AND b.
- OR: De Morgan + NOTs + Toffoli. NOT: X. FANOUT (basis data): CNOT into |0>.

5. Garbage cleanup

- Gate-by-gate simulation uses workspace qubits and leaves intermediate values g(x) (“garbage”).
- Garbage can spoil interference when the computation is a quantum subroutine.
- Clean pattern: compute R → copy useful output → uncompute R†.

```
|x>|0^k>|y> -> |x>|0^k>|y XOR f(x)>
```

- If the classical circuit has t gates, the clean reversible construction has O(t) gate overhead in the lesson.

6. Qiskit quick tools

```
qc.size() qc.depth()
qc.count_ops()
qc.ccx(a,b,t) qc.cx(c,t)
Statevector.from_instruction(qc)
transpile(qc, basis_gates=["h","t","tdg","cx"])
```

NUMBER THEORY & SCALING

7. Representative costs

```
Addition O(n)
Multiplication O(n^2)
Division O(n^2)
GCD O(n^2)
Modular exponent. O(n^3)
Trial factoring O(n^2 2^(n/2))
```

Slides also give number field sieve: 2^{O(n^(1/3)(lg n)^(2/3))}.

8. Algorithms to remember

- Euclid: repeat (a,b) ← (b, a mod b) until b=0.
- Repeated squaring: read exponent bits and reduce modulo M at every multiplication.
- Trial division: test divisors to sqrt(N); for n-bit N, sqrt(N) is about 2^(n/2).
- Slides: Shor gives polynomial-cost quantum factoring.

9. Practice checklist

- Build NAND with CCX + X.
- Make same-size circuits with different depths.
- Build a clean 3-bit majority oracle.
- Trace Euclid and repeated squaring.
- Measure trial-division tests versus bit length.

10. Study flow

Encode → **Build** → **Measure** → **Analyze** → **Reversibilize** → **Uncompute**

Use this loop for each new classical subroutine you want to place inside a quantum algorithm.

Common traps

- Confusing integer magnitude with bit length.
- Using gate count when depth is the real bottleneck.
- Forgetting to restore ancillas/workspace to |0>.
- Treating CNOT FANOUT as arbitrary-state cloning.